Field Hours

CSCE 636 Deep Learning

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

A prediction, a target, and a loss

You predict with y equals w times x plus b and score the half square loss, one half times (y minus t) squared. That one half makes the derivative in y equal to (y minus t), not twice that. For row x equals 1 and t equals 1, w equals 0.5 and b equals 0 yield y equals 0.5. You may change w only because the loss falls. A fit on 1 row is not evidence about the next row.

Bias is a weight on a constant input of 1. You can fold b into the same rule as w by appending a 1 to x. With two weights, a miss can be blamed on either coordinate, and the gradient tells you the split. If x is 0, the weight w gets gradient 0 and only b can move. Check x equals 0: w stays put, while b can still cut a loss of 0.125.

Prediction, then one scalar loss.

1. Compute y from w and b

2. Subtract the target

3. Square and multiply by 1/2

4. Keep the row for the next test

One row, loss 0.125

1. x is 1, w is 0.5, b is 0, and t is 1. Then y is 0.5 times 1 plus 0, so 0.5.

2. The residual is 0.5 minus 1, so minus 0.5. Half of the square is 0.5 times 0.25, which is 0.125.

3. That 0.125 is the number a later step is allowed to reduce. Fitting this single row does not tell you the loss on a second row.

Problem 1. x is 1, w is 0.5, b is 0, and t is 1. What are y and the half square loss?

Answer. y is 0.5 and the loss is 0.125. The residual is minus 0.5, its square is 0.25, and half of that is 0.125.

Problem 2. x is 0 and the loss is still wrong. Which parameter can move, and which gets gradient 0?

Answer. b can move, because bias is a weight on a constant input of 1. w gets gradient 0, since this input did not touch the loss.

Lecture 2 · 90 minutes

The derivative that names the blame

You fill the gradient one partial at a time. For half square loss, the partial of the loss in w is (y minus t) times x,

and the partial in b is (y minus t) times 1. Compute y, then the residual, then multiply. On the row x equals 1, y equals 0.5, t equals 1, the residual is minus 0.5, so the w partial is minus 0.5 times 1. You read that sign before you step.

Backpropagation is this multiplication on a bigger graph. If y goes through a second unit, you multiply by that unit's local derivative as you walk backward. A linear second unit with slope 2 multiplies every upstream partial by 2. You still start from the residual at the loss, which is one number, and you never differentiate the data. Write 0 into a partial to freeze that weight while a new block of 1 layer adapts.

Residual times the local factor.

Form the residual -> Multiply by x for w -> Multiply by 1 for b -> Walk backward for more units

A partial of minus 0.5

1. x is 1, y is 0.5, and t is 1. The residual y minus t is minus 0.5.

2. The partial in w is (y minus t) times x, so minus 0.5 times 1, which is minus 0.5. The partial in b is minus 0.5 times 1, also minus 0.5.

3. A second linear unit of slope 2 would multiply that upstream partial by 2, producing minus 1. You still do not differentiate x. x is data.

Problem 1. y is 0.5, t is 1, and x is 1. What is the partial of the half square loss with respect to w?

Answer. Minus 0.5. The residual is minus 0.5, and you multiply by x, which is 1.

Problem 2. A second linear unit has slope 2. What does backpropagation multiply the upstream partial by?

Answer. 2. You multiply by the local derivative as you walk backward, and you still start from the residual at the loss.

Lecture 3 · 90 minutes

One parameter step, then another

You step each weight by minus the learning rate times its partial. Rate 0.1 and a w partial of minus 0.5 add 0.05 to w, so 0.5 becomes 0.55. The new y on x equals 1 is 0.55, the residual is minus 0.45, and the half square loss falls from 0.125 to 0.10125. A rate of 10 can raise the loss, so you test the rate on a batch of 32.

Momentum stores a velocity. With coefficient 0.9, start at 0 and set velocity to 0.9 times 0 plus a partial of minus 0.5, so velocity is minus 0.5. You then subtract rate times velocity. At rate 0.1 the first step sends w from 0.5 to 0.55. The second step keeps part of that velocity, 0.9 times minus 0.5, and can move farther than the new partial alone. Try this on 1 weight before you trust it on a large net.

A step you recompute before the next one.

1. Scale the partial by the rate

2. Subtract from the weight

3. Recompute y and the loss

4. Reject a rate that raises it

From 0.5 to 0.55

1. The partial in w is minus 0.5 and the rate is 0.1. The update is minus the rate times the partial, so minus 0.1 times minus 0.5, which is plus 0.05.

2. w moves from 0.5 to 0.55. On x equals 1 the new y is 0.55 and the residual is 0.55 minus 1, so minus 0.45.

3. The new half square loss is 0.5 times 0.45 squared. 0.45 squared is 0.2025, and half of that is 0.10125. The loss fell from 0.125. A rate of 10 would add 5 to w instead and can raise the loss.

Problem 1. w is 0.5, the partial is minus 0.5, and the rate is 0.1. What is the new w, and what is the new half square loss on x equals 1 with t equals 1?

Answer. w is 0.55 and the loss is 0.10125. The new prediction is 0.55, the residual is minus 0.45, and half of 0.2025 is 0.10125.

Problem 2. Momentum coefficient 0.9, velocity starts at 0, partial is minus 0.5. What is the velocity on the first step, and does w land in a different place than plain descent at rate 0.1?

Answer. The velocity is minus 0.5. w still goes from 0.5 to 0.55. The first step matches plain descent. The carried term 0.9 times minus 0.5 shows up on the second step.

Lecture 4 · 90 minutes

A small kernel slid over a patch

Convolution shares one kernel across patches. You multiply kernel entries by the inputs under them and add. A 2 by 2 kernel on a 3 by 3 input, stride 1, no padding, yields a 2 by 2 map. You store 4 weights, not a separate weight for each of the 4 outputs. Use it when the same local pattern can appear anywhere in the 3 by 3 input.

You backpropagate into the kernel by summing, over patches, the upstream residual times the input under that entry. One patch with residual 2 and input 3 contributes 6. A pattern seen in 1 of 4 patches still changes the detector used everywhere. Do not invent input when padding is 0. An edge output has fewer patches in the sum, and you must count them or the kernel gradient is scaled wrong.

One kernel, many patches, one update.

Forward patch

Multiply, then add

Output of this patch is 2

4 weights, not 4 sets

Kernel partial

Residual times input

One patch contributes 6

Sum over every patch

Products that sum to 2

1. The kernel, row by row, is 1, 0, 0, 1. The patch under it is 1, 2, 0, 1.

2. The products are 1 times 1, 0 times 2, 0 times 0, and 1 times 1, so 1, 0, 0, and 1. The output is 2.

3. Stride 1 and no padding on a 3 by 3 input give a 2 by 2 map. All 4 outputs share these 4 weights. A residual of 2 on a patch whose input under the first entry is 3 contributes 6 to that entry.

Problem 1. Kernel 1, 0, 0, 1 on patch 1, 2, 0, 1. What is the output?

Answer. 2. The products are 1, 0, 0, and 1.

Problem 2. One patch has upstream residual 2 and the input under a kernel entry is 3. What does that patch add to the entry's partial?

Answer. 6. You multiply residual by the input that touched the entry, then you still have to sum the other patches.

Lecture 5 · 90 minutes

A state carried to the next input

A recurrent state carries a summary forward. Use h equals 0.5 times the previous h plus the input, starting at 0. Inputs 1 then 1 give h equals 1 and then 1.5, so the older input survives at half strength. To train the 0.5 you walk backward through both steps, multiplying by 0.5 at each hop. Two hops contribute a factor of 0.25 to the older input's partial.

A forget gate between 0 and 1 scales the old state before you add the new input. Gate 0 wipes h. Gate 1 copies it, so a distractor input of 9 stays out if the input gate is also 0. You now have 4 maps instead of 1, and you should still hand-check a sequence of length 2. A length of 40 is the same recurrence, not a new idea.

The older input is scaled at every hop.

1. Start h at 0

2. Scale h by 0.5

3. Add the new input

4. Multiply by 0.5 walking back

Two inputs, state 1.5

1. h starts at 0. The first input is 1, so h becomes 0.5 times 0 plus 1, which is 1.

2. The second input is 1, so h becomes 0.5 times 1 plus 1, which is 1.5. The first input remains as 0.5 inside that sum.

3. Backpropagation through both hops multiplies by 0.5 twice. The factor on the older input is 0.25. A forget gate of 0 would have wiped the 0.5 instead of keeping it.

Problem 1. h starts at 0 and the multiplier is 0.5. Inputs are 1 then 1. What are the two states?

Answer. 1, then 1.5. The second state is 0.5 times 1 plus 1.

Problem 2. You walk backward through 2 hops that each multiply by 0.5. What factor reaches the older input?

Answer. 0.25. Two factors of 0.5 multiply. That is why a chain of 20 steps can shrink an early gradient.

Lecture 6 · 90 minutes

Freeze most of the net, train the top

You freeze the first 4 layers by setting their rate to 0, and you train a new top layer at 0.1. Those frozen layers hold features from the old task. A top vector of 10 scores can still fit your labels. Unfreezing everything at 0.1 can wipe the features, since the new loss ignores the old task. After the top layer settles, a fine-tune at 0.01 lets the lower layers move a little without a large jump.

After one batch of 16, the frozen block's partial norm should be 0, or you must skip its step. Computing a partial and still applying it is not freezing. The loss can fall while those weights drift. Compare one weight before and after a single step. If it moved, your rate-0 story is false. You only learn about adaptation when the 4 frozen layers stay unchanged and the top layer is the one that changed.

A frozen block is a claim you can falsify.

Frozen 4 layers

Rate set to 0

Weight identical after 1 step

Partial norm 0, or skip

New top layer

Rate 0.1 at first

10 class scores

Later fine-tune at 0.01

One weight that was supposed to sit still

1. You record a weight in layer 2. It is 1.25. The rate on the first 4 layers is supposed to be 0.

2. After one step on a batch of 16 the weight is 1.27. It moved, so the freeze failed even if the training loss fell.

3. The top layer of 10 scores is allowed to change at rate 0.1. A later fine-tune at 0.01 may move layer 2, but only after you meant to unfreeze it.

Problem 1. A weight in a frozen layer goes from 1.25 to 1.27 in one step while the loss falls. Did the freeze work?

Answer. No. A frozen weight must stay put. A falling loss does not prove the rate was 0. Compare the weight before and after the step.

Problem 2. Why fine-tune at 0.01 instead of unfreezing all 4 layers at the original rate 0.1?

Answer. Rate 0.1 can erase features the new loss does not care about. After the top layer of 10 scores has settled, 0.01 lets the lower layers move a little.

Lecture 7 · 90 minutes

The same loss on three kinds of input

A vision example can be 32 by 32, that is 1024 gray values, with 10 class scores at the top. A speech frame can be 40 log mel bins, or 200 numbers with 2 frames of context on each side. A control state can be 8 ranges plus 1 angle, with 3 command outputs. You train each with the same loss idea and a different tensor. Feeding the 40-bin frame into the vision kernel is not transfer.

Speech and control need memory: a recurrent state, or a stack of 5 frames. A single picture often needs no recurrence if the label is that picture alone. You can freeze 4 vision layers and retrain a top of 10 scores. A control policy may have nothing to freeze and must learn from plus 10 and minus 1. If you have 100 labeled images and 0 labeled traces, do not pretend the same adaptation plan fits both.

Same training step, three shapes.

1024 pixels, 10 scores

40 bins, or 200 with context

8 ranges plus 1 angle

3 commands or a class

200 speech inputs is not 1024 pixels

1. A vision tensor is 32 times 32, so 1024 values. A speech frame with 2 frames of context on each side is 5 times 40, so 200 values.

2. You do not reshape 200 into 1024. The kernel was tied to the image grid. The shapes are different models.

3. Control can be 8 ranges plus 1 angle, 9 numbers, with 3 outputs. If you have 100 labeled images and 0 labeled traces, only the vision top has a supervised set to adapt.

Problem 1. A frame has 40 bins and you add 2 frames on each side. How many inputs is that, and why is it not a 32 by 32 image?

Answer. 200. Five frames times 40 is 200. A 32 by 32 image is 1024 values, and a vision kernel is not a model of a 40-bin spectrum.

Problem 2. You have 100 labeled images and 0 labeled wheel traces. Which adaptation plan is available?

Answer. Freeze 4 vision layers and retrain a top of 10 scores. There is no labeled trace set, so you cannot pretend the control policy has the same fine-tune.

Lecture 8 · 90 minutes

A training step you can audit

You can audit a training step by hand. From x equals 1 and w equals 0.5 you form y, the half square loss, and the partial in w. At rate 0.1 you move w to 0.55 when that partial is minus 0.5. You can share a 2 by 2 kernel, carry a state scaled by 0.5, and freeze 4 layers. A tensor of 40 bins does not belong in a 32 by 32 net.

One check should combine the kernel and the step. Suppose a kernel entry of 1 sees input 3 and the upstream residual is 2, so its partial is 6. A rate of 0.1 moves that entry to 1 minus 0.6, which is 0.4. If you also froze that layer, the entry must stay 1. You are done when the kernel update and the freeze agree on that entry.

The same partial, two different updates.

1. Partial is residual times input

2. Here that product is 6

3. Rate 0.1 moves 1 to 0.4

4. A freeze leaves the entry at 1

Entry 1, partial 6, two outcomes

1. The kernel entry is 1. The input under it is 3 and the upstream residual is 2. The partial is 2 times 3, so 6.

2. At rate 0.1 you subtract 0.1 times 6, which is 0.6. The entry becomes 1 minus 0.6, so 0.4.

3. If the layer is one of the 4 frozen layers, you skip the step. The entry stays 1. A loss that fell does not override that check.

Problem 1. x is 1, w is 0.5, the partial in w is minus 0.5, and the rate is 0.1. What is the updated w?

Answer. 0.55. You add 0.05, which is minus the rate times a partial of minus 0.5.

Problem 2. Combine the kernel gradient with a freeze. Entry 1 sees input 3, the residual is 2, and the rate is 0.1. What is the entry if you step, and what is it if the layer is frozen?

Answer. If you step, the partial is 6 and the entry becomes 0.4. If the layer is frozen, the entry stays 1. Both checks use the same partial.